

Question 1: Browser Back Button History

Scenario:

You are developing a simplified web browser for a small startup. The browser needs a "Back" button feature that lets users return to previously visited web pages in reverse order (the most recently visited page should appear first when the user clicks "Back"). The marketing team also wants a feature that shows the current page the user is on.

Task:

Implement a Stack in C++ to manage the browsing history. Your program should support the following operations:

1. **visitPage(string url)** – User visits a new web page (push the URL onto the stack).
2. **goBack()** – User clicks the Back button. Display and remove the most recently visited page. If there is no page to go back to, display "No previous page available".
3. **currentPage()** – Display the URL of the page the user is currently viewing (top of stack) without removing it.
4. **clearHistory()** – Clear all browsing history (Private Mode).
5. **displayHistory()** – Display all visited pages from most recent to oldest.

Constraints:

- Maximum history capacity: 100 pages (if using array).
- Handle stack overflow through exception.

Sample Input/Output:	
1. Visit Page	Enter choice: 1
2. Go Back	Enter URL: google.com
3. Show Current Page	Page visited: google.com
4. Clear History	Enter choice: 1
5. Display History	Enter URL: youtube.com
6. Exit	Page visited: youtube.com
	Enter choice: 2
	Going back from: youtube.com
	Now on: google.com

Question 2: YouTube Music Playlist Manager

Scenario:

You have been hired by a startup that is building a lightweight music streaming app similar to YouTube Music. The development team has asked you to implement the playlist management module. Users should be able to create their own playlist by adding songs from YouTube, remove songs they no longer like, insert a newly released song at the very top of their playlist, search for a specific song by its title, and play songs one by one in the order they were added. Since this is a custom data structure module, the team has strictly prohibited the use of STL containers.

Task:

Implement a Singly Linked List in C++ to manage a YouTube music playlist. Each node should store:

- Song ID (integer)
- Song Title (string)
- Artist Name (string)
- Duration in seconds (integer)

Your program should support the following operations:

1. **addSong()** – Add a new song to the end of the playlist (user appends a song they like).
2. **addNewRelease()** – Insert a newly released song at the beginning of the playlist so the user sees it first.
3. **removeSong(int id)** – Remove a song from anywhere in the playlist using its Song ID (when the user dislikes or removes it).
4. **searchSong(string title)** – Search for a song by its title and display its details
5. **displayPlaylist()** – Display all songs currently in the playlist in order.
6. **countSongs()** – Display the total number of songs in the playlist.

Constraints:

- Use a singly linked list with dynamic memory allocation (new / delete).
- Handle edge cases: empty playlist, song not found, removing the head/tail node.
- Properly free memory when nodes are removed to avoid memory leaks.

Sample Input/Output:	Enter choice: 1 ID: 101, Title: A, Artist: B, Duration: 204 Song added to playlist.
1. Add Song	
2. Add New Release (Top)	Enter choice: 2 ID: 202, Title: C Artist: D, Duration: 175 New release added at the top.
3. Remove Song	
4. Search Song	
5. Display Playlist	Enter choice: 5
6. Count Songs	1 -> [202] C by D
7. Exit	2 -> [101] A by B